

EMERGENCY RESPONSE NETWORK USING WIRELESS SENSOR MESH NETWORKS

Wireless Sensor Networks Project Report

BATCH NO- 19 GROUP NO- 34

PRANAVI RAJU-SE23UARI060
UPPALA SIDDHARTTH-SE23UCSE241
VADDE ASHWIN-SE23UCAM030
SAMHITH REDDY-SE23UMCS034

Abstract

This project presents an Emergency Response Network based on a Wireless Sensor Mesh Network designed for disaster management scenarios such as fire, earthquake, and mass casualty incidents. The system simulates a distributed network of sensor nodes that continuously monitor environmental and physiological conditions. These nodes generate alerts based on abnormal readings and transmit them through a mesh network to a central base station.

The system incorporates intelligent routing using Dijkstra's shortest path algorithm and a flooding-based fallback mechanism to ensure reliable communication. A key feature of the system is its self-healing capability, where the network automatically reconfigures itself when nodes fail, maintaining uninterrupted communication. Alerts are prioritised using a severity-based scoring system and processed in a priority queue, ensuring critical emergencies are handled first.

The simulation demonstrates how wireless sensor mesh networks can provide resilient, scalable, and efficient communication in disaster scenarios where traditional infrastructure fails.

INTRODUCTION

In disaster situations such as earthquakes, fires, and large-scale accidents, traditional communication systems often fail due to infrastructure damage or overload. This leads to delays in emergency response, which can result in severe consequences, including loss of life and property.

Therefore, there is a need for a communication system that is independent, reliable, and capable of functioning under harsh conditions.

Wireless Sensor Networks (WSNs) provide an effective solution for such scenarios. A WSN consists of distributed sensor nodes that can collect data and communicate wirelessly. When organised in a mesh topology, these nodes can forward data through multiple paths, ensuring communication even when some nodes fail.

This project implements a simulation of a **Disaster Response Mesh Network**, where sensor nodes detect environmental changes and generate alerts. The system is capable of routing data efficiently and recovering from failures through a self-healing mechanism.

OBSERVATION

The system simulates a smart emergency communication network that performs the following functions:

- Creates a wireless mesh network of sensor, relay, and base station nodes
- Continuously monitors environmental conditions such as temperature, smoke, seismic activity, and heart rate
- Detects disaster scenarios like fire, earthquake, and mass casualty
- Generates alerts based on severity levels
- Transmits alerts to a base station using efficient routing algorithms
- Uses shortest-path routing (Dijkstra) for efficiency
- Uses flooding as a fallback mechanism if routes fail
- Automatically reconfigures the network when nodes fail (self-healing)
- Maintains communication even in adverse conditions

In simple terms, the system acts like a **distributed emergency monitoring and communication system that ensures alerts always reach the control centre, even when parts of the network are damaged.**

SYSTEM ARCHITECTURE

3. System Architecture

The system architecture is divided into five main modules:

1. Network topology module
2. Sensor node module
3. Sensor data generation module
4. Alert engine module
5. Routing and self-healing module

The project uses a simulated mesh network made of **15 nodes**.

These include **10 sensor nodes, 4 relay nodes, and 1 base station**. The network is built as a graph using NetworkX, where nodes represent physical devices and edges represent wireless communication links. Nodes are connected if they are within a predefined signal range. This allows the system to simulate a real wireless sensor network deployed in a disaster area.

3.1 Sensor Nodes

Sensor nodes are the main data collection devices in the network. Each sensor node represents a physical WSN device that can collect readings from the environment. In this project, sensor nodes collect four main types of data:

- Temperature
- Smoke level
- Seismic activity
- Heart rate

Each sensor node has a battery level and a state. The node can be in one of three states:

ACTIVE

The node is working normally and can sense data and communicate with other nodes.

SLEEP

The node enters low-power mode when its battery becomes low. This helps conserve energy.

FAILED

The node is no longer functional. This may represent hardware failure, disaster damage, or complete battery loss.

The node module also simulates battery drain. Sensor nodes drain battery slowly, relay nodes drain more because they forward more traffic, and the base station drains the least.

3.2 Relay Nodes

Relay nodes are intermediate nodes that help forward data from sensor nodes to the base station. In a disaster area, some sensor nodes may be far from the base station and may not be able to communicate directly. Relay nodes help bridge this gap.

Relay nodes are important because they:

- Improve network coverage
- Reduce communication distance between nodes
- Help maintain connectivity
- Support multi-hop routing

In this project, relay nodes are placed strategically between sensor clusters and the base station so that communication can continue even when direct paths are not available.

3.3 Base Station

The base station acts as the central command center. It receives alerts from the sensor network and represents the emergency monitoring unit.

The base station performs the following role:

- Receives emergency alerts
- Acts as destination node for routing
- Represents command/control center
- Helps analyze network survivability

In the simulation, node 14 is used as the base station. Other nodes route emergency information toward this base station.

3.4 Mesh Network Topology

A mesh network allows nodes to connect with multiple neighboring nodes. Unlike a star network, where every node depends on one central point, a mesh network allows alternate paths. This is highly useful in disaster management because if one node fails, the system can still use other nodes to transmit information.

In this project:

- Nodes are placed on a simulated geographic grid.
- Distance between nodes is calculated.
- If two nodes are within signal range, an edge is created.

- The graph stores nodes, links, and link weights.
- Link weight represents the communication cost between nodes.

The system automatically builds the network topology when the simulation starts. The topology includes nodes and edges, and this forms the base for routing and failure recovery.

SENSOR AND ALERT SYSTEM

4. Sensor Data Generation

The sensor module generates readings depending on the disaster scenario. The system supports different emergency scenarios such as normal conditions, fire, earthquake, and mass casualty.

Each sensor reading contains:

- Sensor type
- Sensor value
- Unit
- Timestamp
- Node ID
- GPS coordinates

This makes every reading self-contained and useful for alert generation and reporting.

4.1 Temperature Sensor

The temperature sensor is mainly used to detect fire conditions. Under normal conditions, the temperature remains in a safe range.

During a fire scenario, the generated temperature becomes very high and crosses the emergency threshold.

For example:

- Normal condition: around 20°C to 35°C
- Fire condition: around 93°C to 120°C

This helps the system identify fire-affected nodes.

4.2 Smoke Sensor

The smoke sensor measures air quality or smoke concentration. Smoke level is represented using AQI units. In fire scenarios, smoke values become very high and trigger emergency alerts.

For example:

- Normal condition: 0 to 100 AQI
- Fire condition: 410 to 500 AQI

High smoke levels combined with high temperature strongly indicate a fire emergency.

4.3 Seismic Sensor

The seismic sensor detects earthquake activity. It generates values in Richter scale units.

For example:

- Normal condition: 0 to 2 Richter
- Earthquake condition: 5 to 8 Richter

This allows the system to simulate earthquake detection. During earthquake scenarios, affected nodes show high seismic values and may also fail during the simulation.

4.4 Heart Rate Sensor

The heart rate sensor is used to simulate medical distress or mass casualty situations. In normal conditions, heart rate remains in a safe range. In mass casualty conditions, heart rate values become unusually high.

For example:

- Normal condition: 60 to 100 BPM
- Mass casualty condition: 130 to 180 BPM

This can represent injured people, emergency medical monitoring, or rescue personnel's health tracking.

ALERT GENERATION SYSTEM

The alert engine processes sensor readings and converts them into alerts. It evaluates the seriousness of the emergency and assigns an alert level.

The alert levels used in the system are:

1. **INFO** – normal or low-risk condition
2. **WARNING** – an early sign of possible danger
3. **CRITICAL** – serious emergency condition
4. **SOS** – highest emergency level requiring immediate response

The alert engine creates alert records, stores them in a database, and manages them using a priority queue. SOS alerts are processed first because they represent the most urgent situations.

5.1 Threshold-Based Classification

Each sensor type has predefined thresholds. If a sensor reading crosses a threshold, it is classified into a higher alert level.

For example:

- Very high temperatures may become SOS.
- High smoke level may become CRITICAL or SOS.
- High seismic value may indicate earthquake risk.
- A high heart rate may indicate a casualty or medical emergency.

The system uses these thresholds to classify readings as INFO, WARNING, CRITICAL, or SOS.

5.2 Weighted Severity Scoring

The system does not depend only on one sensor. Instead, it combines readings from all sensors into a single severity score from 0 to 100.

The weights are:

Sensor	Weight
Smoke level	30%
Temperature	25%
Seismic activity	25%
Heart rate	20%

Smoke has the highest weight because smoke is a strong indicator of fire or hazardous air conditions. Temperature and seismic activity also have high importance. Heart rate is used to detect medical distress.

The final score is mapped to an alert level:

0–24	INFO
25–49	WARNIN G
50–74	CRITICAL
75–100	SOS

This makes the alert system more reliable because it considers the combined emergency condition rather than only one reading.

5.3 Priority Queue for Alerts

The alert engine uses a priority queue. In a priority queue, the most important alert is processed first.

Priority order:

1. SOS
2. CRITICAL
3. WARNING
4. INFO

This is useful in emergency response because rescue teams must respond to the most serious alerts first. For example, if the system receives 10 INFO alerts and 1 SOS alert, the SOS alert should not wait behind the INFO alerts. The priority queue ensures that emergency alerts are handled in the correct order.

ROUTING AND SELF-HEALING

6. Routing Mechanism

Routing is the process of finding a path for data to travel from a source node to a destination node. In this project, the source is usually a sensor node that detects an emergency, and the destination is the base station.

For example, if node 3 detects fire, it must send the alert to node 14, which is the base station. The message may not travel directly. Instead, it may pass through relay nodes or other sensor nodes.

The routing engine in this project uses two routing methods:

1. Dijkstra shortest path routing
2. Flooding fallback routing

The routing module also simulates latency for each hop and maintains a route discovery log.

6.1 Dijkstra Shortest Path Algorithm

Dijkstra's algorithm is used as the primary routing method. It finds the shortest or lowest-cost path between two nodes in the network graph.

In this project:

- Nodes represent devices.
- Edges represent communication links.
- Edge weight represents link cost.
- Dijkstra finds the path with minimum total cost.

For example, a route may look like:

Node 3 → Node 10 → Node 11 → Node 14

This means node 3 sends data to node 10, node 10 forwards it to node 11, and node 11 forwards it to the base station.

Dijkstra is efficient because it avoids unnecessary broadcasting. It chooses the best available path and reduces delay.

6.2 Flooding Fallback Routing

Flooding is used when Dijkstra fails. Dijkstra may fail if:

- A node fails
- A link breaks
- The graph becomes disconnected
- The source or destination is unreachable

In flooding, the message is sent to all neighboring nodes. Those neighbors then forward the message further until the destination is found.

Flooding is less efficient than Dijkstra because it creates more network traffic. However, it is useful as a backup method because it increases the chance of message delivery when the normal route is broken.

6.3 Hybrid Routing Strategy

The system uses a hybrid strategy:

First: Try Dijkstra shortest path

If Dijkstra succeeds: Use that path

If Dijkstra fails: Use flooding as fallback

This gives the system both efficiency and reliability. Under normal conditions, shortest-path routing is used. Under failure conditions, flooding helps maintain communication.

A good report line:

“The proposed system uses a hybrid routing mechanism where Dijkstra’s algorithm provides efficient shortest-path routing, while flooding is used as a fallback to improve reliability during node failures.”

7. Self-Healing Mechanism

Self-healing is one of the most important features of this project. In disaster situations, nodes may fail because of physical damage, battery drain, or communication loss. A normal network may stop working if an important node fails. But in this project, the mesh network tries to repair itself automatically.

When a node fails:

1. The node is marked as FAILED.
2. It is removed from the network graph.
3. Its affected neighbors are identified.
4. New routes are calculated from affected nodes to the base station.
5. If a new path exists, the system records a self-healing event.

This mechanism allows the network to continue operating even after failures.

7.1 Example of Self-Healing

Suppose the original route is:

Node 7 → Node 9 → Node 13 → Node 14

If node 9 fails, the old route becomes invalid.

The network then tries to find another path, such as:

Node 7 → Node 8 → Node 12 → Node 14

This new route allows communication to continue. This is called self-healing because the network automatically adapts without manual repair.

7.2 Importance of Self-Healing in Emergency Response

Self-healing is very important in disaster management because the network may be deployed in unstable environments. In such situations, it is not practical to manually repair every failed node immediately. A self-healing system improves:

- Reliability
- Fault tolerance
- Network survivability
- Emergency communication continuity

This makes the project suitable for emergency response applications.

SIMULATION, RESULTS, CONCLUSION

8. Simulation Scenarios

The project simulates three major emergency scenarios:

1. Fire
2. Earthquake
3. Mass casualty

Each scenario runs for 30 simulation ticks. A tick represents one time step in the simulation. During every tick:

- Node batteries are updated.
- Sensor readings are generated.
- Alerts are created.
- Routes are checked.
- Node statuses are updated.
- Failures may be injected.
- Self-healing may occur.

The simulation runner executes all three scenarios and prints a final summary, including alert totals, failed nodes, self-healing events, and routing log entries.

8.1 Fire Scenario

In the fire scenario, nodes 3, 4, and 5 are affected. These nodes generate high temperature and high smoke readings.

Expected behavior:

- Temperature rises above normal values.
- Smoke level becomes very high.
- Fire-related alerts are generated.

- Many alerts may reach SOS level.
- Alerts are sent toward the base station.

This scenario shows how the system detects fire using environmental sensors.

8.2 Earthquake Scenario

In the earthquake scenario, nodes 7, 8, and 9 are affected. These nodes generate high seismic readings. The simulation also injects node failures during this scenario.

Expected behavior:

- Seismic activity increases.
- Earthquake alerts are generated.
- Some nodes fail during the simulation.
- The routing engine recomputes paths.
- Self-healing events are recorded.

This scenario tests both disaster detection and network resilience.

8.3 Mass Casualty Scenario

In the mass casualty scenario, nodes 0, 1, 2, 3, 4, and 6 are affected. These nodes generate high heart-rate readings.

Expected behavior:

- Heart rate values become abnormally high.
- Medical distress alerts are generated.
- Alerts are prioritized based on severity.
- The system continues monitoring all active nodes.

This scenario represents emergency health monitoring in a disaster area.

DASHBOARD AND BACKEND

The project also includes a Flask-based backend. The backend runs the simulation and sends live updates using SocketIO.

The dashboard/backend provides:

- Current scenario information
- Node status
- Battery level
- Failed node information
- Alert events
- Topology changes
- Self-healing events
- Network survivability statistics

The backend exposes API routes such as `/api/topology`, `/api/alerts`, and `/api/dispatch/<alert_id>`. These allow the frontend to fetch topology data, view alerts, and update alert status.

This makes the system more interactive and closer to a real emergency monitoring dashboard.

RESULTS

The simulation shows that the system can successfully detect different emergency conditions and generate appropriate alerts.

Main observations:

1. Fire detection works effectively

High temperature and smoke readings generate serious alerts.

2. Earthquake detection is supported

High seismic readings are detected and classified.

3. Mass casualty monitoring is possible

Abnormal heart rate values generate medical emergency alerts.

4. Alerts are prioritised correctly

SOS alerts are processed before lower-level alerts.

5. Routing works using shortest paths

Dijkstra's algorithm finds efficient paths to the base station.

6. Fallback routing improves reliability

Flooding is available when shortest-path routing fails.

7. Self-healing improves network survivability

When nodes fail, the network attempts to find alternate routes.

8. Database storage supports record keeping

Alerts are stored in SQLite for later viewing and analysis.

9. Dashboard support improves monitoring

Real-time status updates help users understand the network condition.

```
app.py
socketio = SocketIO(app, cors_allowed_origins="*", async_mode="threading")

SCENARIOS = [
    {
        "name": "FIRE",
        "icon": "🔥",
        "description": "High temperature and smoke detected in sector 3-5",
        "key": "fire",
        "affected": [3, 4, 5],
        "fail_on_ticks": {},
        "ticks": 30,
    },
    {
        "name": "EARTHQUAKE",
        "icon": "🌋",
        "description": "Seismic activity detected - nodes 7-9 at risk",
        "key": "earthquake",
        "affected": [7, 8, 9],
        "fail_on_ticks": {8: [9], 16: [7]},
        "ticks": 30,
    },
    {
        "name": "MASS CASUALTY",
        "icon": "👤",
        "description": "Multiple casualties detected across sectors 0-4, 6",
        "key": "mass_casualty",
        "affected": [0, 1, 2, 3, 4, 6],
        "fail_on_ticks": {12: [11]},
        "ticks": 30,
    },
]

_lock = threading.Lock()
sim_state = {
    "network": None,
    "scenario": "INITIALIZING",
    "scenario_payload": None, # last scenario_change payload, replayed on conn
    "tick": 0,
    "heals": 0,
    "total_alerts": 0,
}
```

```
sensor.py
sensor.py - Sensor data generators for each disaster scenario.
Provides both per-node helpers and batch generation across a node set.
Each reading is a self-contained dict with value, unit, timestamp,
node_id, and gps_coors so alerts/engine.py can be fully self-sufficient.

import random
from datetime import datetime

# Threshold definitions (used by alerts/engine.py as well)

THRESHOLDS = {
    "temperature": {
        "WARNING": 45.0, # °C
        "CRITICAL": 65.0,
        "SOS": 90.0,
    },
    "smoke_level": {
        "WARNING": 150.0, # AQI
        "CRITICAL": 250.0,
        "SOS": 400.0,
    },
    "seismic_activity": {
        "WARNING": 2.5, # Richter
        "CRITICAL": 4.0,
        "SOS": 6.0,
    },
    "heart_rate": {
        "WARNING": 110.0, # BPM
        "CRITICAL": 140.0,
        "SOS": 165.0,
    },
}
```

ADVANTAGES OF PROPOSED SYSTEM

The proposed system has several advantages:

- It does not fully depend on fixed infrastructure.
- It supports multi-hop communication.
- It detects multiple types of emergencies.
- It prioritises critical alerts.
- It can recover from node failures.
- It simulates battery-aware sensor nodes.
- It provides real-time monitoring through a backend/ dashboard.
- It is modular and can be extended in the future.

LIMITATIONS

Although the system works well as a simulation, it has some limitations:

- It is not tested on real hardware.
- The sensor readings are simulated.
- GPS locations are fixed.
- Wireless interference is not fully modelled.
- Energy consumption is simplified.
- Real-world obstacles are not considered.
- Security and encryption are not implemented.

These limitations can be addressed in future work.

FUTURE SCOPE

The project can be improved in the following ways:

- 1. Hardware implementation**

The system can be implemented using real IoT devices such as Arduino, Raspberry Pi, ESP32, or LoRa modules.

- 2. Drone-based relay nodes**

Drones can be used as temporary relay stations in damaged areas.

- 3. AI-based prediction**

Machine learning can be added to predict disaster severity based on sensor trends.

- 4. Cloud integration**

Alerts can be sent to cloud platforms for large-scale monitoring.

- 5. Mobile application**

A mobile app can be created for rescue teams.

- 6. Security features**

Encryption and authentication can be added to protect emergency data.

- 7. Larger network simulation**

The number of nodes can be increased to simulate a city-level emergency network.

- 8. Real map integration**

The system can be integrated with Google Maps or OpenStreetMap for live location tracking.

CONCLUSION

This project successfully demonstrates an Emergency Response Network using Wireless Sensor Mesh Networks. The system is designed to monitor disaster conditions, generate alerts, prioritise emergencies, and maintain communication even when some nodes fail.

The use of mesh topology makes the system suitable for disaster environments because it provides multiple paths for communication. The routing mechanism uses Dijkstra's algorithm for efficient path selection and flooding as a backup method. The self-healing feature allows the network to automatically reroute data after node failures, improving reliability and survivability.

The simulation of fire, earthquake, and mass casualty scenarios shows that the system can handle different emergency conditions. The alert engine ensures that serious alerts, such as SOS, are processed first. The backend and dashboard support real-time monitoring, making the system practical for emergency response visualisation.

Overall, the project shows how wireless sensor networks can be used to build reliable, fault-tolerant, and rapidly deployable emergency communication systems.